VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:

Trần Vinh Quang	2550342
Lê Hiển Vinh	2453422
Nguyễn Đức Thiên Tân	2550345
Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026

Contents

1 Introduction

1.1 Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

1.2 Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

1.3 Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using `Verible` and the SystemVerilog Language Server (`SVLS`) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom `Python` script is implemented with Icarus Verilog (`iverilog`) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

1.4 Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

1.5 Report Organization

This report is organized into six main sections.

Section 1 introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section 2 presents the theoretical background of the project. It explains the basic concepts of RISC architecture, instruction format, CPU execution cycle, datapath, control path, and supported instructions.

Section 3 describes the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section 4 discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section 5 presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section 6 concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

2 Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is `INST_ADDR`). Program execution stops upon encountering the `HLT` instruction.

By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table 2.1: CPU Controller state and Control signals

Outputs	**Phase**								**Notes**
	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

2.1 Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (`sel = 1`) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (`rd = 1`) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the `ld_ir = 1` control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

2.2 Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (`sel = 0`). Simultaneously, the PC increments (`inc_pc = 1`) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory (`rd = 1`).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is `SKZ` and the `zero` flag condition is met, the PC increments once more (`inc_pc = 1`) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the `STO` instruction, asserting `wr = 1` and `data_e = 1`) or loaded into the Accumulator (for arithmetic and logic instructions, asserting `ld_ac = 1`).

3 Design

3.1 Top-level System Architecture

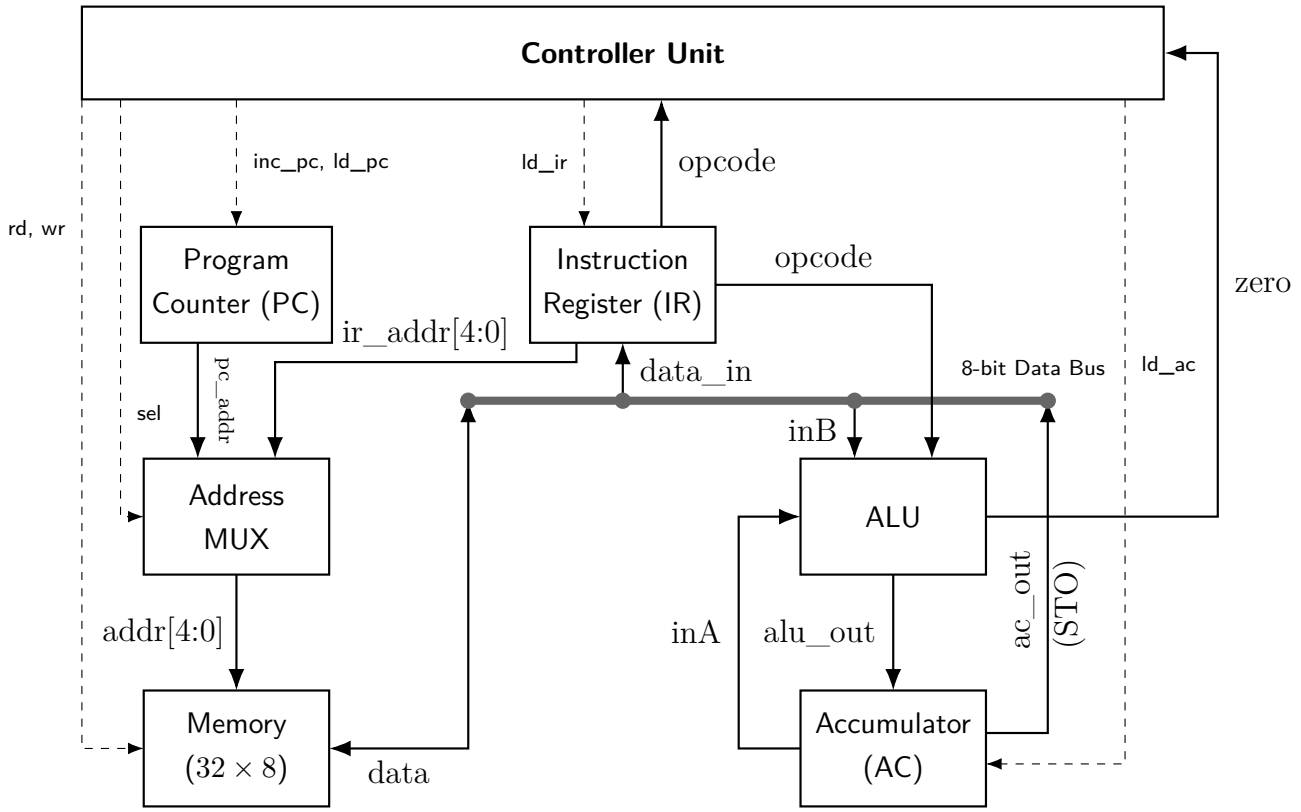

Figure 3.1: Top-Level Architecture of the RISC Processor

3.2 Functional Blocks

3.2.1 Program Counter (PC)

- **Function:**
  - The counter is a crucial component used to track and count program instructions.
  - It can also be utilized to count program states.
  - The counter operates synchronously on the rising edge of the `clk` signal.
  - The `reset` signal is active-high, clearing the counter back to 0.
  - The counter has a width of 5 bits.
  - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

3.2.2 Address Multiplexer (Address MUX)

- **Function:**
  - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
  - The MUX has a default data width of 5 bits.
  - The width is parameterized to allow for easy modification if necessary.
- **Inputs:** `pc_addr`, `ir_addr`, `sel`.
- **Outputs:** `addr_out`.

3.2.3 Arithmetic Logic Unit (ALU)

- **Function:**
  - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
  - The ALU executes 8 distinct operations on 8-bit operands (`inA` and `inB`).
  - It produces a 8-bit output and a 1-bit asynchronous `zero` flag.
  - The `zero` flag indicates whether the input `inA` is equal to 0.

- **Inputs:** `inA`, `inB`, `opcode`.
- **Outputs:** `alu_out`, `zero`.

Table 3.1: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
`HLT`	000	Halts program execution.	`inA`
`SKZ`	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	`inA`
`ADD`	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	`inA + inB`
`AND`	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	`inA AND inB`
`XOR`	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	`inA XOR inB`
`LDA`	101	Reads the value from the address in the instruction and loads it into the Accumulator.	`inB`
`STO`	110	Writes the Accumulator's data to the memory address specified in the instruction.	`inA`
`JMP`	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	`inA`

3.2.4 Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** `clk`, `rst`, `opcode`, `zero`.
- **Outputs:** `sel`, `rd`, `ld_ir`, `halt`, `inc_pc`, `ld_ac`, `ld_pc`, `wr`, `data_e`.

Table 3.2: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

3.2.5 Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** clk, rst, ld_ir, data_in.
- **Outputs:** opcode, operand.

3.2.6 Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** clk, rst, ld_ac, data_in.
- **Outputs:** ac_out.

3.2.7 Memory

- **Function:**
  - The memory module stores both instructions and data.
  - It is designed to separate read and write functions while utilizing a single bidirectional data port. Reading and writing cannot occur simultaneously.
  - Configured for 5-bit addresses and 32-bit data.
  - Uses 1-bit control signals to enable reading (rd) and writing (wr).
  - Memory operations are synchronized to the rising edge of the clk signal.
- **Inputs:** clk, rd, wr, addr.
- **Inout:** data.

4 Implementation

```
module PC (
    input              clk,
    input              rst,
    input              ld_pc,
    input              inc_pc,
    input      [4:0] data_in,
    output reg [4:0] pc_out
);

  always @(posedge clk) begin
    if (rst) begin
      pc_out <= 5'd0;
    end else if (ld_pc) begin
      pc_out <= data_in;
    end else if (inc_pc) begin
      pc_out <= pc_out + 5'd1;
    end else begin
      pc_out <= pc_out;
    end
  end

endmodule
```

Listing 4.1: Program Counter Module

```
module address_mux #(
    parameter WIDTH = 5
) (
    input  [WIDTH-1:0] pc_addr,
    input  [WIDTH-1:0] ir_addr,
    input              sel,
    output [WIDTH-1:0] addr_out
);
    assign addr_out = (sel) ? pc_addr : ir_addr;
endmodule
```

Listing 4.2: Address MUX Module

```
module ALU (
    input [2:0] opcode,
    input [7:0] inA,
    input [7:0] inB,
    output reg [7:0] alu_out,
    output zero
);
  always @(*) begin
    case (opcode)
      3'b000:  alu_out = inA;          //HLT
      3'b001:  alu_out = inA;          //SKZ
      3'b010:  alu_out = inA + inB;    //ADD
      3'b011:  alu_out = inA & inB;    //AND
      3'b100:  alu_out = inA ^ inB;    //XOR
      3'b101:  alu_out = inB;          //LDA
      3'b110:  alu_out = inA;          //STO
      3'b111:  alu_out = inA;          //JMP
      default: alu_out = 8'b00000000;
    endcase
  end
  assign zero = (inA == 8'b00000000);
endmodule
```

Listing 4.3: ALU Module

```
module controller (
    input clk,
    input rst,
    input [2:0] opcode,
    input zero,

    output reg sel,
    output reg rd,
    output reg ld_ir,
    output reg halt,
    output reg inc_pc,
    output reg ld_ac,
    output reg ld_pc,
    output reg wr,
    output reg data_e

);
  localparam  INST_ADDR=3'd0,
              INST_FETCH=3'd1,
              INST_LOAD=3'd2,
              IDLE=3'd3,
              OP_ADDR=3'd4,
              OP_FETCH=3'd5,
              ALU_OP=3'd6,
              STORE=3'd7;

  // Opcodes
  localparam HLT = 3'b000,
             SKZ = 3'b001,
             ADD = 3'b010,
             AND = 3'b011,
             XOR = 3'b100,
             LDA = 3'b101,
             STO = 3'b110,
             JMP = 3'b111;

  reg [2:0] state;
  reg halted;
```

```
39
40  wire aluop;
41  assign aluop = (opcode == ADD) ||
42                 (opcode == AND) ||
43                 (opcode == XOR) ||
44                 (opcode == LDA);
45
46  // State register
47  always @(posedge clk) begin
48    if (rst) begin
49      state <= INST_ADDR;
50      halted <= 1'b0;
51  end else if (halted) begin
52      state  <= state;
53      halted <= 1'b1;
54    end else begin
55      if ((state == OP_ADDR) && (opcode == HLT)) begin
56        state  <= OP_ADDR;
57        halted <= 1'b1;
58      end else begin
59        state <= (state == STORE) ? INST_ADDR : state + 3'd1;
60      end
61    end
62  end
63
64  // Output logic (combinational)
65  always @(*) begin
66    // Default values
67    sel    = 1'b0;
68    rd     = 1'b0;
69    ld_ir  = 1'b0;
70    halt   = 1'b0;
71    inc_pc = 1'b0;
72    ld_ac  = 1'b0;
73    ld_pc  = 1'b0;
74    wr     = 1'b0;
75    data_e = 1'b0;
76
77    if (rst) begin
```

```
78        sel = 1'b1;
79      end else if (halted) begin
80        halt = 1'b1;
81      end else begin
82        case (state)
83
84          INST_ADDR: begin
85            sel = 1'b1;
86          end
87
88          INST_FETCH: begin
89            sel = 1'b1;
90            rd  = 1'b1;
91          end
92
93          INST_LOAD: begin
94            sel   = 1'b1;
95            rd    = 1'b1;
96            ld_ir = 1'b1;
97          end
98
99          IDLE: begin
100           sel   = 1'b1;
101           rd    = 1'b1;
102           ld_ir = 1'b1;
103         end
104
105         OP_ADDR: begin
106           inc_pc = 1'b1;
107
108           if (opcode == HLT) begin
109             halt = 1'b1;
110           end
111         end
112
113         OP_FETCH: begin
114           rd = aluop;
115         end
116
```

```
117         ALU_OP: begin
118           rd = aluop;
119
120           if (opcode == SKZ && zero) begin
121             inc_pc = 1'b1;
122           end
123
124           if (opcode == JMP) begin
125             ld_pc = 1'b1;
126           end
127
128           if (opcode == STO) begin
129             data_e = 1'b1;
130           end
131         end
132
133         STORE: begin
134           rd    = aluop;
135           ld_ac = aluop;
136
137           if (opcode == JMP) begin
138             ld_pc = 1'b1;
139           end
140
141           if (opcode == STO) begin
142             wr     = 1'b1;
143             data_e = 1'b1;
144           end
145         end
146
147         default: begin
148           sel    = 1'b0;
149           rd     = 1'b0;
150           ld_ir  = 1'b0;
151           halt   = 1'b0;
152           inc_pc = 1'b0;
153           ld_ac  = 1'b0;
154           ld_pc  = 1'b0;
155           wr     = 1'b0;
```

```
156             data_e = 1'b0;
157           end
158
159         endcase
160       end
161     end
162
163 endmodule
```

Listing 4.4: Controller Unit Module

```
module instruction_register (
    input         clk,
    input         rst,
    input         ld_ir,
    input  [7:0] data_in,
    output [ 2:0] opcode,
    output [ 4:0] operand
);
  reg [7:0] ir_reg;
  always @(posedge clk) begin
    if (rst) begin
      ir_reg <= 8'b00000000;
    end
    else if (ld_ir) begin
      ir_reg <= data_in[7:0];
    end
  end
  assign opcode  = ir_reg[7:5];
  assign operand = ir_reg[4:0];
endmodule
```

Listing 4.5: Controller Unit Module

```
module accumulator (
    input       clk,
    input       rst,
    input [7:0] data_in,
    input       ld_ac,
    output reg [7:0] ac_out
);
  always @(posedge clk) begin
    if (rst) ac_out <= 8'b00000000;
    else if (ld_ac) ac_out <= data_in;
    else ac_out <= ac_out;
  end
endmodule
```

Listing 4.6: Accumulator Register Module

```
module Memory (
    input        clk,
    input        rd,
    input        wr,
    input [4:0] addr,
    inout [7:0] data
);

  reg [7:0] mem_cells[31:0];
  reg [7:0] data_out;

  always @(posedge clk) begin
    if (wr && !rd) begin
      mem_cells[addr] <= data;
    end else
    if (rd && !wr) begin
      data_out <= mem_cells[addr];
    end
  end

  assign data = (rd && !wr) ? data_out : 8'hZZ;

endmodule
```

Listing 4.7: Memory Module

```
module CPU (
    input  clk,
    input  rst,
    output halt
);

  // Control signals
  wire sel;
  wire rd;
  wire ld_ir;
  wire inc_pc;
  wire ld_ac;
  wire ld_pc;
  wire wr;
  wire data_e;

  // Datapath signals
  wire [4:0] pc_out;
  wire [4:0] operand;
  wire [4:0] mem_addr;

  wire [2:0] opcode;

  wire [7:0] data_bus;
  wire [7:0] ac_out;
  wire [7:0] alu_out;

  wire zero;

  // Program Counter
  PC u_pc (
      .clk    (clk),
      .rst    (rst),
      .ld_pc  (ld_pc),
      .inc_pc (inc_pc),
      .data_in(operand),
      .pc_out (pc_out)
  );
```

```
39
40  // Address MUX
41  address_mux #(
42      .WIDTH(5)
43  ) u_address_mux (
44      .pc_addr (pc_out),
45      .ir_addr (operand),
46      .sel     (sel),
47      .addr_out(mem_addr)
48  );
49
50  // Memory
51  Memory u_memory (
52      .clk (clk),
53      .rd  (rd),
54      .wr  (wr),
55      .addr(mem_addr),
56      .data(data_bus)
57  );
58
59  // Instruction Register
60  instruction_register u_ir (
61      .clk    (clk),
62      .rst    (rst),
63      .ld_ir  (ld_ir),
64      .data_in(data_bus),
65      .opcode (opcode),
66      .operand(operand)
67  );
68
69  // Accumulator
70  accumulator u_ac (
71      .clk    (clk),
72      .rst    (rst),
73      .data_in(alu_out),
74      .ld_ac  (ld_ac),
75      .ac_out (ac_out)
76  );
77
```

```
78    // ALU
79    ALU u_alu (
80        .opcode (opcode),
81        .inA    (ac_out),
82        .inB    (data_bus),
83        .alu_out(alu_out),
84        .zero   (zero)
85    );
86
87    // Controller
88    controller u_controller (
89        .clk   (clk),
90        .rst   (rst),
91        .opcode(opcode),
92        .zero  (zero),
93
94        .sel   (sel),
95        .rd    (rd),
96        .ld_ir (ld_ir),
97        .halt  (halt),
98        .inc_pc(inc_pc),
99        .ld_ac (ld_ac),
100       .ld_pc (ld_pc),
101       .wr    (wr),
102       .data_e(data_e)
103   );
104
105 endmodule
```

Listing 4.8: CPU Module

5 Testing and Evaluation

6 Conclusion and Future Work

6.1 Conclusion

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction

format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

Key part of the design was the Controller, which was implemented as a finite state machine with eight states: `INST_ADDR`, `INST_FETCH`, `INST_LOAD`, `IDLE`, `OP_ADDR`, `OP_FETCH`, `ALU_OP`, and `STORE`. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

6.2 Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the

current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.

7 Member Contribution

Table 7.1: Member Contribution

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiển Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed the testbenches and expected output files, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero flag, and register operations. Contributed to the project report.